

GO PROGRAMMING MASTER COURSE

Week 6, Day 1

Functions and methods inside SMS — extract handlers from `main`, keep health green.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE LAST
WEEK

A running API is the starting line

- GET /health returns 200 with the exact JSON. If not, fix that first.
- Cold-call: what is a handler? What does Gin wrap?
- Today we **refactor**, not add Postgres or JWT.
- gofmt still wins. Unformatted PRs returned.

TODAY

If Day 1 of week 6 works, you can

- Name the parts of a function: name, params, results.
- Return (value, error) and never silently ignore err.
- Use defer for cleanup at function exit.
- Tell a function from a method; preview pointer receivers.
- Move health/hello into internal/handlers; keep :8080 green.
- Ship `FullName(first, last string)` string with a test.

**SOFTWARE
ENGINEERING**

Refactor is a feature when the layout was lying

main that owns every route will rot. Handlers belong in a package with one job: talk HTTP.

- SMS milestone: extract, do not expand the product surface.
- PR description says what moved and why — not “fixed stuff”.
- Green health before and after is the proof.

FUNCTIONS

Parts of a function

```
func FullName(first, last string) string {  
    return first + " " + last  
}
```

```
func Max(nums []int) (int, bool) {  
    // name · params · results  
}
```

■ **Name** — exported if capital (FullName).

■ **Parameters** — typed; same type can share: first, last string.

■ **Results** — zero, one, or many. Named results: use sparingly.

PASS
BY
VALUE

Arguments are copies — unless you share memory

```
func bump(n int)
func main() {
    x := 3
    bump(x)
    fmt.Println(
}
```

■ Ints, structs (by default), and other values are copied into the function.

■ Slices and maps copy the *header* but share the backing data — week 3 gotcha returns.

■ Methods that must change a struct usually use a **pointer receiver** (preview).

MULTIPLE
RETURNS

return x, err is the Go handshake

```
func ParsePort(s string) (
    if s == "" {
        return 0, fmt.Errorf("empty string")
    }
    // ...
    return 8080, nil
}

port, err := ParsePort(os.Args[1])
if err != nil {
    log.Fatal(err)
}
```

Ignoring err with _ is a bug unless you can explain why.

Handlers will return JSON errors — still check err from callees.

DEFER

Run this when the function exits

```
func work() {  
    f, err := os.Open("notes.txt")  
    if err != nil { return }  
    defer f.Close() // runs at e  
  
    // read f ...  
}
```

- LIFO: last defer runs first.
- Use for close, unlock, and later recover patterns.
- In HTTP handlers, prefer returning; defer shines with files and mutexes (week 12).

METHODS

A function with a receiver

```
type HealthHandler struct{}

func (h *HealthHandler) Get(c *gin.Context) {
    c.JSON(http.StatusOK, gin.H{
        "status": "ok", "message": ""
    })
}

h := &HealthHandler{}
r.GET("/health", h.Get)
```

Same idea as a plain function — plus a home type for state later (repo, config).

Course target: (h *StudentHandler) style methods in internal/handlers.

CONSTRUCTORS

New... is just a function that returns a value

```
type HelloHandler struct{}

func NewHelloHandler() *HelloHandler {
    return &HelloHandler{}
}

func (h *HelloHandler) Get(c *Context) {
    c.JSON(http.StatusOK, g
}
```

You do not fully need struct theory yet — copy the shape.

Later
NewStudentHandler(repo)
will inject dependencies.

POINTER VS
VALUE ·
PREVIEW

Why handlers use `*HealthHandler`

Value receiver

`func (h HealthHandler) —`
method gets a copy. Fine for
tiny immutable helpers.

Pointer receiver

`func (h *HealthHandler)`
— shared instance; can hold
repo/db later.

Course default

HTTP handlers: pointer
receivers. Matches this repo.

Deep dive

Stack/heap and escape analysis
— week 8. Memorise the
pattern today.

GENERICs · ONE
EXAMPLEThen
stop

```
func First[T any](s []T) (T,  
    var zero T  
    if len(s) == 0 {  
        return zero, false  
    }  
    return s[0], true  
}
```

```
v, ok := First([]string{"SE"
```

- One generic helper so the syntax is not alien in docs.
- Do not build a generics library. SMS does not need it this week.

SMS
MILESTONE

Extract week 5 into internal/handlers

```
backend/  
  cmd/server/main.go  
  internal/handlers/  
    health.go  
    hello.go  
    name.go
```

- main stays thin. Routes can stay in main or a tiny routes register.
- Health JSON contract must not change.

LIVE
DEMO

REFACTOR

Move health out of main

```
// internal/handlers/health.go
package handlers
func NewHealthHandler() *HealthHandler { return &HealthHandler{} }
func (h *HealthHandler) Get(c *gin.Context) { /* c.JSON(200, "OK") */ }
```

```
// cmd/server/main.go
h := handlers.NewHealthHandler()
r.GET("/health", h.Get)
r.Run(":8080")
```

LIVE
DEMO

HELPER

FullName for later student responses

```
// internal/handlers/rAdd name_test.go.go test
package handlers      ./internal/handlers must be
func FullName(first, l green.
    first = strings.Tr
    last = strings.Tri
    if first == "" { r
    if last == "" { re
    return first + " "
```

WIRING

What main should look like after

```
func main() {  
    r := gin.Default()  
    health := handlers.NewHealthHandler()  
    hello := handlers.NewHelloHandler()  
    r.GET("/health", healthHandler)  
    r.GET("/api/hello", helloHandler)  
    r.Run(":8080")  
}
```

- No business logic in main beyond wiring.

- Optional:
internal/routes.Register(r)
if the file stays tidy.

IGNORING
ERRORS

_ is honest; silence is not

```
// bad - you hid a failure  
json.Marshal(x)
```

```
// ok only if you truly do not care  
_, _ = fmt.Println("note")
```

```
// usual  
b, err := json.Marshal(x)  
if err != nil { /* handle */ }
```

- Blank identifier means “I see this value; I choose not to use it.”

- Reviewers will ask about ignored error returns.

Done when all of this is true

- Health + hello still 200 on :8080
- Handler logic lives under `internal/handlers`
- `FullName` + green go test `./internal/handlers`
- `main` mostly wires routes
- PR opened describing the refactor

HOMEWORK

PR the refactor — no new product feature

- Required: handlers package + thin main + green tests for FullName.
- README: note the new layout in one paragraph.
- Stretch: internal/routes registers paths; main only Run.
- Do not add JWT, Postgres, or student CRUD this week.

NEXT
SESSION

Week 7 — structs, JSON tags, in-memory students

- `models.Student`, create + list via Postman.
- JSON tags matching this repo.
- Bring a clean handlers layout and green health.

WORDS TO
KEEP

Say these correctly next week

Function

Named code with params
and results.

Method

Function with a receiver
type.

defer

Runs at function exit (LIFO).

Constructor

NewX () function returning
a ready value.

Instructor merges when

- gofmt clean; health JSON unchanged
- Handlers not still pasted entirely inside main
- `go test ./internal/handlers` green
- PR says what moved — not a zip dump

SAFETY

Still true while refactoring

- No secrets in git.
- Do not “fix” by deleting go.mod.
- Keep port :8080 unless the whole class agrees otherwise.
- Commit working health before a big move — small steps.

Questions